

Concurrency

Introduzione

Definizione — Concorrenza (concurrency): significa eseguire più compiti "contemporaneamente". La lezione distingue tre casi che spesso vengono confusi:

- **Elaborazione parallela (parallel processing):** quando hai più CPU o un processore multicore, e i compiti girano *davvero* nello stesso istante. L'esempio dei tre chef che cucinano ciascuno una portata dello stesso pasto nello stesso momento serve proprio a questo: tre "processori" (chef) che lavorano in parallelo.
- **Multitasking (con una sola CPU):** in senso stretto non è parallelismo. È un solo chef che prepara il pasto di tre portate scambiandosi rapidamente da un compito all'altro. Sembra simultaneo ma in realtà è un unico esecutore che alterna velocemente.
- **Multithreading:** è anch'esso concorrenza. Parti separate dello stesso programma vengono eseguite da thread indipendenti.

Vantaggi della concorrenza:

- *Prestazioni migliori* — ma con un'avvertenza importante che il docente sottolinea: i tre chef producono il pasto solo alla velocità del più lento dei tre. Cioè il parallelismo aiuta, ma il collo di bottiglia resta l'elemento più lento.
- *Tempi di risposta più rapidi* — esempio: un programma con un thread per l'interfaccia utente e un altro per il database resta reattivo verso l'utente anche mentre il database lavora.

Svantaggi della concorrenza:

- *Risorse condivise:* se più thread condividono uno stesso dato, quel dato va protetto dalla corruzione (è il tema centrale della seconda metà del corso: race condition).
- *Deadlock, livelock e starvation* — problemi che verranno affrontati alla fine.

Terminologia (definizioni precise date a lezione):

- **Processo:** è costituito da uno o più thread.
- **Thread:** è la più piccola unità di esecuzione; esegue i compiti.
- **Task (compito):** definisce il *lavoro* che il thread eseguirà.

Concetto chiave — non determinismo: l'ordine di esecuzione delle operazioni **non è garantito**. Solo perché avvii un thread per primo, non significa che verrà eseguito per primo. Lo **scheduler** ha una logica interna che *non è sotto il tuo controllo*. L'unica cosa quasi garantita (in assenza di bug) è che un thread avviato verrà prima o poi portato a termine. Tutto il resto è imprevedibile. Questo è un punto che ritornerà continuamente in tutta la trattazione.

Creating Threads (Creazione dei thread)

Ci sono diversi modi per creare un thread in Java. La lezione ne mostra quattro.

Modo 1 — Estendere la classe Thread:

La tua classe diventa una sottoclasse di `Thread`. Sovrascrivi il metodo `run()` inserendovi la tua logica personalizzata. Poiché sei un sottotipo di `Thread`, puoi chiamare direttamente metodi come `getName()`.

- **Svantaggio (motivo per cui è poco popolare):** una volta che estendi `Thread`, non puoi estendere nessun'altra classe (Java non ha ereditarietà multipla di classi).

```
class MioThread extends Thread {  
  
    @Override  
    public void run() {  
        System.out.println("Nome thread: " + getName());  
    }  
}  
  
// nel main:  
MioThread t = new MioThread();  
t.start(); // FONDAMENTALE: start(), non run()
```

Punto critico ripetuto a lezione: si chiama **`start()`**, non `run()`. `start()` crea effettivamente un nuovo thread e chiama internamente il costruttore/`run()`. Nel main c'è sempre il **thread principale (main)** che è quello che avvia tutto; il tuo nuovo thread prende un nome tipo `Thread-0`.

Modo 2 — Implementare l'interfaccia `Runnable`:

`Runnable` è un'interfaccia funzionale con un unico metodo astratto: `run()` (senza argomenti, tipo di ritorno `void`). Metti la tua logica in `run()`. Qui *non* sei un sottotipo di `Thread`, quindi per ottenere il nome del thread corrente usi il metodo statico `Thread.currentThread().getName()`. Passi la tua istanza `Runnable` al costruttore di `Thread` e poi chiami `start()`.

```
class MioTask implements Runnable {  
  
    @Override  
    public void run() {  
        System.out.println(Thread.currentThread().getName());  
    }  
}  
  
// nel main:  
Thread t = new Thread(new MioTask());  
t.start();
```

Questo modo è preferito perché **più flessibile** (la tua classe resta libera di estendere altro).

Modo 3 — Runnable con lambda:

Poiché Runnable è un'interfaccia funzionale, puoi passare direttamente una lambda al costruttore di Thread, senza scrivere una classe separata.

```
Thread t = new Thread() ->
    System.out.println(Thread.currentThread().getName())
);
t.start();
```

In tutti e tre i casi l'output è **non deterministico**.

Errore da evitare — chiamare run() direttamente:

Se invece di start() chiami direttamente run(), **non viene creato alcun nuovo thread**. Il metodo run() viene eseguito nel thread corrente (cioè nel main), nello stack di chiamate corrente. Quindi non vedrai mai un Thread-0: tutto gira nel thread principale. Questo è il punto che il docente vuole farti "apprezzare".

Sleep() and join()

Due metodi popolari della classe Thread, spiegati con un programma di **conto alla rovescia** (un array che conta da 9 a 0, e alla fine deve stampare "boom").

Problema di partenza: senza controlli, il conto alla rovescia stampa tutto istantaneamente (nessun ritardo) e per giunta il "boom" compare all'inizio invece che alla fine. Due problemi da risolvere: (1) mettere un ritardo tra i numeri, (2) far comparire "boom" alla fine.

Definizione — sleep(): mette il thread corrente "a dormire" per un certo numero di millisecondi. Serve a dare agli *altri* thread la possibilità di girare (a non far sì che un thread entri e resti lì finché non ha finito tutto). Nel programma, dopo aver stampato ogni numero il thread dorme 1000 ms (mezzo secondo... in realtà 1000 ms = 1 secondo), così il conto alla rovescia diventa cadenzato.

Punto tecnico: sleep() può lanciare InterruptedException (un thread che dorme può essere interrotto), quindi va racchiuso in un blocco try/catch.

```
public void run() {
    for (int i = 9; i >= 0; i--) {
        System.out.println(i);
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

```
}  
}
```

Questo risolve il problema (1) — il ritardo — ma il "boom" è ancora all'inizio. Perché? Perché il thread principale, dopo aver fatto `start()` sul thread timer, **prosegue subito** e stampa "boom" senza aspettare che il conto alla rovescia finisca.

Definizione — `join()`: fa sì che il thread chiamante (qui il main) **attenda** che un altro thread (qui il thread del timer) sia terminato prima di proseguire. "Unirsi alla fine del thread del timer."

```
Thread timer = new Thread(new CountdownRunnable());  
timer.start();  
try {  
    timer.join(); // il main aspetta qui finché timer non finisce  
} catch (InterruptedException e) {  
    e.printStackTrace();  
}  
System.out.println("boom"); // ora esce alla fine
```

Anche `join()` può lanciare `InterruptedException`, quindi anch'esso va in `try/catch`.

Riepilogo del docente: `sleep()` è stato usato per forzare il thread del conto alla rovescia ad andare a dormire mezzo secondo dopo ogni numero (dando accesso al processore agli altri thread e ritardando l'output); `join()` è stato usato per mantenere il thread principale in attesa fino al termine del thread timer, così che "boom" esca davvero alla fine.

ExecutorService — introduzione

Il problema che risolve: l'API della concorrenza (`ExecutorService`) **astrae la gestione dei thread**. Ti permette di eseguire elaborazioni complesse che coinvolgono thread **senza doverli gestire direttamente** (niente più `new Thread(...).start()` a mano).

Definizioni:

- **ExecutorService:** un'interfaccia che fornisce servizi per creare e gestire thread.
- **Executors:** una classe di utilità con metodi statici che restituiscono implementazioni di `ExecutorService`.
- **Pool di thread (thread pool):** un insieme di thread riutilizzabili, disponibili per eseguire i task. Invece di creare/distuggere thread di continuo, li riusi.

Tipi di executor disponibili:

1. **Single thread pool executor** (`newSingleThreadExecutor()`): un solo thread; i task vengono elaborati **in modo sequenziale**, nell'ordine in cui arrivano.
2. **Cached thread pool executor** (`newCachedThreadPool()`): i thread vengono creati *quando servono* e riutilizzati quando si liberano. I thread rimasti inattivi troppo a lungo

vengono eliminati. **Attenzione:** in processi di lunga durata il numero di thread può crescere moltissimo.

3. **Fixed thread pool executor** (`newFixedThreadPool(n)`): un numero **fisso** di thread, specificato all'inizio. Un valore tipico è il numero di CPU del sistema, ottenibile con `Runtime.getRuntime().availableProcessors()`.

```
ExecutorService es1 = Executors.newSingleThreadExecutor();
```

```
ExecutorService es2 = Executors.newCachedThreadPool();
```

```
int cpu = Runtime.getRuntime().availableProcessors();
```

```
ExecutorService es3 = Executors.newFixedThreadPool(cpu);
```

Come si inviano i task — Runnable vs Callable:

Definizione — Callable: simile a Runnable, ma con due differenze *critiche*: un Callable può restituire un risultato e può lanciare un'eccezione controllata (checked).

Confronto preciso fatto a lezione:

- Entrambi sono asincroni, rappresentano task eseguiti dai thread, ed entrambi sono interfacce funzionali con un metodo astratto.
- Runnable: il metodo `run()` **non restituisce nulla** (void) e **non** lancia eccezioni controllate.
- Callable<V> (generico, tipizzato su V): il metodo `call()` **restituisce V** e **può lanciare un'eccezione**.

Metodi per inviare i task:

- `execute(...)`: accetta un Runnable (non è sovraccaricato per Callable).
- `submit(...)`: può funzionare sia con Runnable sia con Callable. Poiché con Callable puoi ottenere un valore di ritorno, di solito si usa `submit()` con un Callable.

Sincrono vs asincrono (analogia dell'officina):

- *Sincrono*: porti l'auto dal gommista e ti siedi ad aspettare che sia pronta.
- *Asincrono*: consegna l'auto, vai a fare la spesa/lavorare, e torni più tardi a ritirarla. Il lavoro procede "in background" mentre tu fai altro.

Quando invii un Callable con `submit()`, l'esecuzione parte in background e il tuo thread prosegue. Serve quindi un modo per recuperare il risultato *quando sarà pronto*: entra in gioco Future.

Definizione — Future: un oggetto che rappresenta il **risultato di un calcolo asincrono**. Fornisce metodi per verificare se il calcolo è completo e per recuperarne il risultato.

Definizione — get(): il metodo per recuperare il risultato dal Future. È una **chiamata sincrona bloccante**: attende (bloccando il thread, se necessario) fino a quando il calcolo non è terminato, poi restituisce il valore. "Bloccare significa sprecare" tempo del thread, quindi va usato con consapevolezza.

ExecutorService — codice

Due esempi pratici.

Esempio con `execute()` + `Runnable`:

Si crea un single-thread executor e si passa un task `Runnable` (una lambda che non restituisce nulla) con `execute()`. Fondamentale: alla fine bisogna chiamare `shutdown()`, altrimenti l'applicazione non termina mai. `shutdown()` fa sì che i task esistenti possano completarsi, ma **non vengono accettati nuovi task**.

```
ExecutorService es = Executors.newSingleThreadExecutor();
es.execute(() -> System.out.println("task Runnable"));
es.shutdown();
```

Esempio con `submit()` + `Callable` + `Future` + `get(timeout)`:

Si crea un single-thread executor, si invia un `Callable` con `submit()` e si ottiene un `Future`. Il `Callable` calcola, ad esempio, $3 + 5$ e restituisce un intero. Poi si chiama `get()` — qui nella versione **con timeout** (`get(500, TimeUnit.MILLISECONDS)`): aspetta al massimo mezzo secondo.

Eccezioni da gestire (multi-catch):

- `TimeoutException` — se il risultato non arriva entro il tempo specificato;
- `ExecutionException` — se il `Callable` stesso ha lanciato un'eccezione durante l'esecuzione;
- `InterruptedException` — se l'attesa viene interrotta.

```
ExecutorService es = Executors.newSingleThreadExecutor();
Future future = es.submit(() -> 3 + 5);
try { Integer risultato = future.get(500, TimeUnit.MILLISECONDS);
System.out.println(risultato);
} catch (TimeoutException | ExecutionException | InterruptedException e){
e.printStackTrace();
} finally {
es.shutdown();
}
```

Submitting Task Collections — codice

Oltre a inviare un singolo task, l'`ExecutorService` permette di inviare **collezioni di task** (ad esempio una `List` di `Callable`) tramite `invokeAny()` e `invokeAll()`.

La demo usa una lista di `Callable` che restituiscono le stringhe "A", "B", "C", "D", provata con due executor: uno single-thread e uno fixed-thread-pool con 4 thread, per mostrare la differenza.

Definizione — `invokeAny()`:

- Esegue in modo **sincrono** e invia un insieme di task.
- Restituisce il risultato del **primo task che si completa; tutti gli altri task vengono annullati**.
- Su un **single-thread** executor, i task partono in ordine, quindi si completa sempre per primo il primo task → viene *sempre* restituito "A".
- Su un **fixed-thread-pool** con 4 thread, i task possono girare su thread separati, quindi il primo a finire può essere "B", "C" o "D" — dipende dallo scheduler (fuori dal nostro controllo). Esegendolo più volte si vedono risultati diversi.

Definizione — `invokeAll()`:

- È anch'essa una chiamata **sincrona**, ma ritorna **solo quando tutti i task sono completati**.
- **L'ordine dei risultati è mantenuto**: il risultato del primo `Callable` va in prima posizione nella lista restituita, il secondo in seconda, ecc. — indipendentemente da quale thread abbia finito prima.
- Iterando sui risultati escono sempre "A", "B", "C", "D" in quell'ordine.

```
List<Callable<String>> callables = List.of(
    () -> "A", () -> "B", () -> "C", () -> "D"
);
// invokeAny: un solo risultato, il primo che finisce
String uno = es.invokeAny(callables);
// invokeAll: tutti i risultati, in ordine
List<Future<String>> tutti = es.invokeAll(callables);
```

Scheduling Tasks (Programmazione dei task)

Oltre ai tre executor già visti (single-thread, cached, fixed), ce ne sono due che permettono di **programmare** l'esecuzione di task in un momento futuro, ed eventualmente di **ripeterli** a intervalli.

Definizione — `ScheduledExecutorService`: un'interfaccia che fornisce metodi per pianificare attività. Si crea con la classe `Executors`, in due varianti:

`newSingleThreadScheduledExecutor()` e `newScheduledThreadPool(n)`.

Entrambe restituiscono uno `ScheduledExecutorService`.

Metodi principali:

- `schedule(...)` — sovraccaricato per `Runnable` e per `Callable`: esegue un task una volta, dopo un certo ritardo.
- `scheduleAtFixedRate(...)` — **tasso fisso**.
- `scheduleWithFixedDelay(...)` — **ritardo fisso**.

Differenza cruciale tra i due (molto sottolineata a lezione):

- **Fixed rate (tasso fisso):** invia i task a intervalli regolari **indipendentemente da quanto ci mettono a completarsi**. Esempio: ritardo iniziale 2000 ms, poi ogni 500 ms → i task partono a 2000, 2500, 3000, 3500 ms... **Pericolo:** se un task impiega *più* del periodo (più di 500 ms), un nuovo task viene "inviato" mentre il precedente non è ancora finito → si accumulano problemi. Va usato con cautela.
- **Fixed delay (ritardo fisso):** aspetta che il task precedente **finisca**, poi attende il periodo indicato, e solo dopo avvia il successivo. Il ritardo è quindi *tra la fine di un task e l'inizio del prossimo*.

```
ScheduledExecutorService ses = Executors.newScheduledThreadPool(10);
```

```
// esecuzione singola dopo 2 secondi
```

```
Future<String> f = ses.schedule(() -> "risultato", 2, TimeUnit.SECONDS);
```

```
// tasso fisso: ogni 500 ms indipendentemente dalla durata
```

```
ses.scheduleAtFixedRate(task, 2000, 500, TimeUnit.MILLISECONDS);
```

```
// ritardo fisso: 300 ms dopo la FINE del task precedente
```

```
ses.scheduleWithFixedDelay(task, ritardoIniziale, 300, TimeUnit.MILLISECONDS);
```

Negli esempi il task stampa l'**ID del thread**: si vede che gli ID (es. 13, 15, 16) vengono **riutilizzati**, dimostrazione che il pool riusa i thread. Il docente nota anche che gli ID iniziano da numeri alti perché molti thread di sistema (es. il garbage collector) girano già in background. Nota pratica: con `scheduleAtFixedRate/WithFixedDelay` senza `shutdown()`, l'applicazione continua a girare finché non la fermi tu.

Thread Safety — Introduzione

Da qui il corso passa alla seconda grande metà: **proteggere i dati condivisi**.

Definizione — il problema: quando quattro (o più) thread accedono a un dato condiviso, possono **corrompersi i dati a vicenda**. Questo è chiamato **data race** (corsa ai dati).

La lezione anticipa le soluzioni che verranno esaminate in ordine:

1. **Classi atomiche** dell'API,
2. la parola chiave **synchronized**,
3. l'interfaccia **Lock**.

Data Race Demo (Dimostrazione della corsa ai dati)

L'esempio: una variabile di classe `count` inizializzata a zero. Un metodo `addToCounter()` fa:

1. legge il valore corrente in una variabile locale `c`,
2. stampa il valore corrente e l'ID del thread,
3. incrementa `c` e lo rimemorizza in `count`.

Il trucco didattico: la stampa (`println`) è **lenta** rispetto alle altre operazioni, e questo dà ai thread l'opportunità di **entrare e interrompersi a vicenda** proprio nel mezzo dell'operazione.

Il main crea **dieci thread**, ognuno chiama `addToCounter()`. In teoria il valore finale dovrebbe essere 10. **In pratica non ci si avvicina nemmeno.**

```
static int count = 0;

void addToCounter() {
    int c = count;           // leggo
    System.out.println("prima: " + c +
        " thread: " + Thread.currentThread().getId());
    c++;                     // incremento (NON atomico)
    count = c;               // riscrivo
    System.out.println("dopo: " + count);
}
```

Perché fallisce: l'operazione "leggi → incrementa → riscrivi" **non è atomica**. Un thread legge `count` (es. 0), viene interrotto prima di riscrivere, un altro thread legge anch'esso 0, entrambi scrivono 1. Un incremento è andato perso. Eseguendolo più volte, il risultato finale è sempre diverso e sempre inferiore a 10. Questa è la **corruzione dei dati** dovuta a una data race.

Atomic Classes (Classi atomiche)

Prima soluzione.

Definizione — operazione atomica: un'operazione **indivisibile**. Nessun thread può entrare e interrompere un altro thread mentre questo incrementa, decrementa o modifica il dato. Precisazione importante del docente: *non* si garantisce che un thread resti sempre "in esecuzione" per tutta la durata; si garantisce che, anche se il thread entra ed esce dallo stato di esecuzione, **nessun altro thread possa agire sullo stesso dato** nel frattempo.

Le tre classi da conoscere: `AtomicInteger`, `AtomicLong`, `AtomicBoolean`. Il focus è su `AtomicInteger`. Si trovano nel package `java.util.concurrent.atomic`.

Metodi principali:

- Comuni a tutte e tre: `get()` (legge il valore corrente), `set(nuovo)` (imposta, come l'operatore di assegnazione), `getAndSet(nuovo)` (imposta il nuovo valore e restituisce il vecchio), `compareAndSet(atteso, nuovo)` (imposta il nuovo valore **solo se** il valore corrente è uguale a quello atteso).
- Per le classi numeriche: `incrementAndGet()` (pre-incremento, come `++x`), `getAndIncrement()` (post-incremento, come `x++`), `decrementAndGet()` (pre-decremento), `getAndDecrement()` (post-decremento).

AtomicInteger example — codice

Si sostituisce l'`int` con un `AtomicInteger` inizializzato a zero. Si usa un `ExecutorService` con cinque thread, si cicla dieci volte inviando task, e ogni task chiama semplicemente `incrementAndGet()` (incrementa di uno e restituisce il valore, atomicamente).

```
AtomicInteger count = new AtomicInteger(0);
ExecutorService es = Executors.newFixedThreadPool(5);
for (int i = 0; i < 10; i++) {
    es.submit(() -> count.incrementAndGet());
}
es.shutdown();
```

Risultato: ora si arriva **sempre a dieci** e **non ci sono duplicati**, perché l'incremento è atomico (inizia e finisce senza che nessun thread interrompa). **Ma** — punto ripetuto — **l'ordine resta non garantito**: le cifre da 1 a 10 escono tutte, ma in ordine sparso.

Limite delle classi atomiche (che introduce il prossimo argomento): proteggono una *singola variabile/operazione*, ma **non permettono di proteggere un intero blocco di codice**.

Synchronized keyword (Parola chiave synchronized)

Il problema che risolve: a volte vuoi che un **blocco di codice** (più righe tra parentesi graffe) sia eseguito da **un solo thread alla volta**.

Definizioni:

- **Sezione critica (critical section):** un blocco di codice mutuamente esclusivo — solo un thread per volta può eseguirlo.
- **Monitor / blocco intrinseco (intrinsic lock):** ogni oggetto in Java possiede un lock incorporato (un "monitor") che entra in gioco automaticamente con i blocchi `synchronized`.

Come funziona: un thread che vuole entrare in un blocco `synchronized` prova ad **acquisire il lock**. Se è libero, lo ottiene ed entra. Ogni altro thread in arrivo deve **attendere**. Quando il primo thread esce dalla sezione critica, il lock **si rilascia automaticamente** e uno dei thread in attesa può acquisirlo ed entrare.

Condizione fondamentale: i thread devono sincronizzare **sullo stesso oggetto**. Se usano oggetti diversi, usano lock diversi, e la protezione **non funziona** (la race condition non viene risolta).

Si può applicare `synchronized` sia ai **metodi** sia ai **blocchi di codice**, sia in versione **d'istanza** sia **statica**:

- **Metodo/blocco d'istanza:** il lock è sul riferimento `this`. Un blocco `synchronized(this) { x++; }` equivale a marcare `synchronized` l'intero metodo d'istanza che contiene solo `x++`.
- **Metodo/blocco statico:** il lock è sul **class literal** (`NomeClasse.class`), cioè sull'oggetto della classe, non su un'istanza. Un metodo statico `synchronized` equivale a `synchronized(NomeClasse.class) { ... }`.

```
class UseCounter {
    private int x;
    static int y;

    // sincronizzazione su un blocco (lock = this)
    void incrementA() {
        synchronized (this) { x++; }
    }

    // equivalente: sincronizza l'intero metodo d'istanza (lock = this)
    synchronized void incrementB() { x++; }

    // statico: lock = class literal
    synchronized static void decrementA() { y--; }
    static void decrementB() {
        synchronized (UseCounter.class) { y--; }
    }
}
```

Synchronized example — codice

Si riprende il programma della data race e lo si "aggiusta" con `synchronized`. Oltre a garantire la correttezza (si arriva sempre a 10), qui **si garantisce anche l'ordine** (1, 2, 3, ... 10) — cosa che `AtomicInteger` *non* faceva.

Il docente mostra **quattro modi** di risolverlo:

1. **Metodo statico `synchronized`:** si aggiunge `synchronized` al metodo statico `addToCounter`. L'intero metodo diventa un'operazione atomica: quando un thread entra, resta dentro fino alla graffa di chiusura, poi rilascia il monitor. Nell'output si vedono coppie di ID uguali ("prima 13" / "dopo 13"): significa che lo stesso thread ha eseguito tutto il metodo senza interruzioni — esattamente ciò che si vuole.

2. **Blocco `synchronized` sul class literal (esplicito):** invece del metodo statico `synchronized` (che sincronizza *implicitamente* sull'oggetto classe), si sincronizza **esplicitamente** con `synchronized(NomeClasse.class) { ... }` avvolgendo tutto il contenuto del metodo. Stesso effetto.
3. **Blocco `synchronized` su un lock dedicato:** si crea un oggetto `Object lock = new Object();` e si scrive `synchronized(lock) { ... }`. Tutto il codice interno diventa mutuamente esclusivo. Un thread prende il lock, esegue; gli altri aspettano; poi lo rilascia. Va bene purché tutti sincronizzino sullo *stesso* oggetto `lock`.
4. **Metodo/blocco d'istanza `synchronized` su `this`:** si rende `addToCounter` un metodo d'istanza e si sincronizza su `this`. **Attenzione (punto cruciale):** poiché ora si sincronizza su `this`, **tutti i thread devono condividere la stessa istanza** della classe, altrimenti ognuno sincronizzerebbe su un `this` diverso (lock diversi) e non funzionerebbe. Quindi nel main si crea **un'unica istanza** e tutti e dieci i thread chiamano `addToCounter()` su quella stessa istanza.

// Modo 3 — lock dedicato

```
Object lock = new Object();
static void addToCounter() {
    synchronized (lock) {
        // sezione critica: leggi, stampa, incrementa, riscrivi
    }
}
```

// Modo 4 — istanza condivisa

```
MiaClasse istanza = new MiaClasse(); // UNA sola, condivisa da tutti i thread
// ogni thread: istanza.addToCounter();
synchronized void addToCounter() { /* ... */ } // lock = this
```

Riepilogo dei quattro modi: metodo statico (sincronizza implicitamente il class literal), blocco esplicito sul class literal, lock dedicato con blocco `synchronized`, e metodo/blocco d'istanza su `this` (che richiede istanza condivisa). Tutti producono output ordinato con ID duplicati (coppie prima/dopo dello stesso thread).

Lock example — codice (interfaccia `Lock`)

Perché serve qualcosa oltre `synchronized`: usare correttamente `synchronized` può essere difficile, specie se i dati sono acceduti da molti metodi, e i **costi in prestazioni** possono essere elevati. L'API della concorrenza offre alternative più semplici e più performanti: l'interfaccia **`Lock`**.

Definizioni e differenze rispetto a `synchronized`:

- Con `synchronized`, un thread viene semplicemente bloccato se un altro possiede già il lock (non può fare altro che aspettare).
- Con `Lock` sei **più flessibile**: se non riesci a ottenere il lock, sei **libero di fare qualcos'altro**.
- Esistono due chiamate:
 - **`lock()`** — chiamata **bloccante** (aspetta finché non ottiene il lock).
 - **`tryLock()`** — chiamata **non bloccante**: ritorna immediatamente un `boolean`.
`true` = lock acquisito con successo; `false` = non è stato possibile ottenerlo (e non aspetti).
- Devi **bloccare esplicitamente** un oggetto che implementa `Lock` (mentre con `synchronized` potevi sincronizzare qualsiasi oggetto).
- Devi **rilasciare esplicitamente** il lock con un `unlock()` quando hai finito — idealmente nel blocco **`finally`**, così viene rilasciato in ogni caso.
- Regola d'oro: **sblocca ogni lock che blocchi** (non bloccare 3-4 volte e sbloccare una volta sola).

ReentrantLock: un'implementazione semplice di `Lock` che supporta la mutua esclusione. Il primo thread che chiama `lock()` ottiene il lock; tutti gli altri che arrivano a quella chiamata si bloccano; quando il primo chiama `unlock()`, gli altri sono liberi di competere per il lock.

Lock example — codice (applicato alla data race)

Si risolve la data race con l'interfaccia `Lock` invece che con `synchronized`. Il codice è quasi identico al precedente, ma:

- Si crea un `ReentrantLock`.
- In `addToCounter()` si chiama `lock.lock()`: il primo thread ottiene il lock ed esegue la sezione critica; gli altri si bloccano su quel lock.
- La sezione critica va in un `try`, e nel `finally` si chiama `lock.unlock()` per rilasciare sempre il lock.

```
Lock lock = new ReentrantLock();
```

```
void addToCounter() {  
    lock.lock();  
    try {  
        // sezione critica: leggi, stampa, incrementa, riscrivi  
    } finally {  
        lock.unlock(); // rilascio garantito  
    }  
}
```

Risultato: il valore sale ordinatamente fino a 10, con ID di thread in coppie (prima/dopo).

Versione con `tryLock()`:

```
if (lock.tryLock()) {           // non bloccante

    try {

        // sezione critica

    } finally {

        lock.unlock();         // sblocca solo se avevi ottenuto il lock

    }

} else {

    System.out.println("Impossibile ottenere il blocco");

    // libero di fare altra elaborazione

}
```

Nella demo, il **primo** thread (es. ID 16) ottiene il lock e produce il risultato; **tutti gli altri** falliscono il `tryLock()` e stampano "Impossibile ottenere il blocco" — dimostrando il comportamento non bloccante.

Concurrent Collections — Introduzione

Il problema delle collezioni normali: le collezioni tradizionali (es. `HashMap`) non sono sicure per l'uso concorrente. L'esempio: una `HashMap` con capitali → paesi (Oslo → Norvegia, Copenaghen → Danimarca); si itera con un `for-each` sul `key-set` e, **durante l'iterazione**, si prova a **rimuovere** una chiave dalla mappa → viene lanciata `ConcurrentModificationException`.

Passando alla versione concorrente (`ConcurrentHashMap`), lo stesso codice **funziona senza eccezione**.

Argomenti che verranno trattati: collezioni `SkipList`, collezioni `CopyOnWrite`, code bloccanti (`BlockingQueue`), collezioni sincronizzate.

Concurrent Collections — API

Panoramica delle classi concorrenti (versioni "thread-safe" delle collezioni note):

- Set/List: `ConcurrentSkipListSet`, `CopyOnWriteArrayList`, `CopyOnWriteArraySet`, `ConcurrentLinkedQueue`, `LinkedBlockingQueue`.
- L'unica interfaccia "nuova" legata alla concorrenza citata è **`BlockingQueue`**; le altre interfacce sono quelle già familiari.
- Per le mappe (che, si ricorda, pur non facendo parte della gerarchia `Collection` sono considerate collezioni): `ConcurrentSkipListMap`, `ConcurrentHashMap`, e l'interfaccia `ConcurrentMap`.

Concurrent Collections — SkipList Collections

Definizione: due classi, `ConcurrentSkipListSet` e `ConcurrentSkipListMap`. La caratteristica chiave è che sono **ordinate** per ordine naturale.

Equivalenze da ricordare:

- `ConcurrentSkipListSet` è la versione concorrente di `TreeSet` (che è ordinato perché è un albero).
- `ConcurrentSkipListMap` è la versione concorrente di `TreeMap`, ordinata per chiave.

```
Set<String> paesi = new ConcurrentSkipListSet<>();
paesi.add("Germania"); paesi.add("Canada"); paesi.add("Australia");
// iterando: Australia, Canada, Germania (ordine alfabetico)
```

```
Map<String,Integer> età = new ConcurrentSkipListMap<>();
età.put("Jack", 12); età.put("Zach", 15); età.put("Ana", 22);
// iterando per chiave: Ana, Jack, Zach (ordine alfabetico)
```

Frase mnemonica del docente: "skip" (list) è per la concorrenza, e queste collezioni sono per essere **ordinate**.

Concurrent Collections — CopyOnWrite Collections

Definizione: due classi, `CopyOnWriteArrayList` e `CopyOnWriteArraySet`. Sono adatte a scenari "**molte letture, poche scritture**", perché ogni scrittura ha un **costo elevato**: ogni volta che la collezione viene modificata (aggiunta o rimozione di un elemento), **viene fatta una copia dell'intera collezione**.

Precisazione: la copia riguarda i **riferimenti** contenuti nella collezione, non lo stato degli oggetti puntati. Quindi puoi cambiare il nome di una persona già dentro la collezione senza scatenare copie; ma quando **aggiungi** o **rimuovi** una persona, allora viene creata una copia.

Comportamento dell'iteratore (il punto centrale): l'iteratore lavora su uno **snapshot** — un'istantanea dello stato dell'array **al momento in cui l'iteratore è stato creato**. Quell'array **non cambia mai** per tutta la vita dell'iteratore. Conseguenza: **nessuna `ConcurrentModificationException`**, ed è possibile modificare la collezione *durante* un for-each senza finire in un loop infinito.

Esempio: parti con `[Brian, Carol]`; dentro un for-each che li stampa, aggiungi un nome. La modifica va su una **copia**, non sulla lista che il for-each sta scorrendo. Quindi il ciclo termina normalmente (vede solo lo snapshot Brian/Carol), mentre la lista "reale" finisce per contenere gli elementi aggiunti.

```
List<String> nomi = new CopyOnWriteArrayList<>();
```

```
nomi.add("Brian"); nomi.add("Carol");  
for (String n : nomi) {  
    System.out.println(n);  
    nomi.add("Kyra"); // modifica una COPIA; il for-each vede solo lo snapshot  
}  
  
// nessuna eccezione, nessun loop infinito
```

Versione Set (CopyOnWriteArraySet): stessa logica di snapshot, ma il set mantiene gli elementi **unici** — i duplicati non vengono aggiunti. Quindi anche se provi ad aggiungere un elemento già presente, la dimensione non cresce.

Conclusione: minimizza le scritture. Se fai molte modifiche, questo tipo di collezione **non è adatto** (le prestazioni diventano un problema per via delle copie ripetute).

Concurrent Collections — BlockingQueue

Definizione: ConcurrentLinkedQueue e LinkedBlockingQueue sono entrambe code thread-safe. La LinkedBlockingQueue aggiunge **metodi bloccanti**: cioè metodi che possono **aspettare** per un certo tempo.

I metodi bloccanti chiave prevedono un **timeout** e una **TimeUnit** (millisecondi, secondi, minuti...):

- **poll(timeout, unit):** recupera e rimuove la testa della coda; se la coda è **vuota**, aspetta fino al tempo specificato.
- **offer(elemento, timeout, unit):** inserisce un elemento; se la coda è **piena**, aspetta fino al tempo specificato.

Comportamento di base della coda (FIFO): si aggiunge in coda, si rimuove dalla testa.

```
BlockingQueue<String> coda = new LinkedBlockingQueue<>();  
coda.offer("rosso"); coda.offer("verde"); coda.offer("blu"); // in coda  
coda.poll(); // rimuove "rosso" (la testa)  
coda.peek(); // guarda la testa senza rimuovere → "verde"
```

// metodi bloccanti con timeout (serve try/catch per InterruptedException)

```
coda.offer("bianco", 100, TimeUnit.MILLISECONDS); // aspetta max 100ms se piena  
String x = coda.poll(200, TimeUnit.MILLISECONDS); // aspetta max 200ms se vuota
```

Poiché durante l'attesa il thread può essere interrotto, i metodi con timeout richiedono di gestire **InterruptedException**.

Concurrent Collections — Synchronized Collections

Definizione: con la classe di utilità `Collections` puoi ottenere una versione **sincronizzata** di una collezione *non* concorrente. Esistono metodi come `synchronizedList`, `synchronizedMap`, `synchronizedSet`, `synchronizedSortedMap`, `synchronizedNavigableMap`, `synchronizedSortedSet`, `synchronizedNavigableSet`.

Quando usarle: se hai già una collezione non concorrente esistente e vuoi convertirla in una versione sincronizzata da condividere tra più thread.

Quando NON usarle: se **fin dalla creazione** sai che ti servirà la concorrenza, usa direttamente le **collezioni concorrenti** (viste sopra), perché hanno **prestazioni migliori**.

Limite importante: le collezioni sincronizzate **lanciano ancora**

`ConcurrentModificationException` se provi a modificarle dentro un ciclo — a differenza delle collezioni concorrenti, che *non* la lanciano.

```
Map<String,String> capitali = new HashMap<>();
Map<String,String> sincronizzata = Collections.synchronizedMap(capitali);
// utilizzabile in ambiente multithread...
// MA: modificarla dentro un for-each → ConcurrentModificationException
```

Threading Problems — Race Condition

Da qui la lezione tratta i **quattro problemi** del threading: (1) **race condition**, (2) **deadlock**, (3) **livelock**, (4) **starvation**. Gli ultimi tre riguardano la *liveness* (vivacità/reattività) di un'applicazione.

Definizione — Race condition (condizione di gara): si verifica quando due o più thread accedono **contemporaneamente** a una risorsa condivisa che invece **dovrebbe** essere acceduta in modo sequenziale.

Esempio concettuale (targhe auto): se la generazione di un numero di targa va in race, potresti generare due auto con la stessa targa (scenario peggiore); esiti "migliori" sarebbero il rifiuto di entrambe, o una con targa e una senza.

Esempio in codice — conto bancario:

- Saldo iniziale: 50 €.
- Metodi: leggere il saldo, ed effettuare un prelievo che aggiusta il saldo.
- **Un solo conto bancario**, e lo stesso `Runnable` viene passato a **entrambi** i thread (John e Mary) → condividono lo stesso conto. Questo è il presupposto della race.
- Ogni thread fa cinque prelievi da 10 € ciascuno. Se il saldo scende sotto zero, il conto va **in scoperto** — cosa che non dovrebbe mai accadere e che segnala la race condition.
- Il metodo `makeWithdrawal` controlla se il saldo $\geq$ importo, poi il thread **dorme 500 ms**, e solo dopo effettua il prelievo. Quel `sleep` serve a **forzare** la race: mentre un thread dorme (dopo aver verificato che il prelievo è possibile), un altro thread entra e verifica

anch'esso — così entrambi credono che il saldo basti, entrambi prelevano, e alla fine il saldo va sotto zero (es. -10).

```
void makeWithdrawal(int amount) {  
    if (balance >= amount) {           // controllo  
        try { Thread.sleep(500); }     // dorme: apre la finestra per la race  
        catch (InterruptedException e) { }  
        balance -= amount;             // prelievo  
    } else {  
        System.out.println("Impossibile prelevare");  
    }  
}
```

Soluzioni (le stesse tecniche già viste):

1. **synchronized sull'intero metodo di prelievo:** il metodo diventa atomico — o si esegue completamente o niente. Un thread entra, prende il lock, controlla il saldo, dorme, preleva ed esce senza interruzioni; nessun altro thread può entrare nel frattempo. Output: John esegue tutta la sua transazione atomicamente, poi Mary tutta la sua, ecc. Niente scoperto.
2. **Lock (ReentrantLock):** si crea un lock, si chiama `lock.lock()` all'ingresso, la logica è la stessa, e nel `finally` si chiama `lock.unlock()` per rilasciare. Un thread ottiene il lock e procede; gli altri aspettano. Risultato: i thread arrivano ordinatamente ed eseguono l'intera transazione in modo atomico.

```
Lock lock = new ReentrantLock();  
void makeWithdrawal(int amount) {  
    lock.lock();  
    try {  
        if (balance >= amount) {  
            Thread.sleep(500);  
            balance -= amount;  
        }  
    } catch (InterruptedException e) {  
    } finally {  
        lock.unlock();  
    }  
}
```

(Nota: nel file la lezione "Deadlock" ripete integralmente il testo della race condition — probabilmente un errore/sovrapposizione nella trascrizione. Il vero contenuto su deadlock, livelock e starvation è quello che segue, tratto dalla lezione finale.)

Threading Problems — Deadlock, Livelock e Starvation

Definizione — Deadlock (stallo): i thread coinvolti sono **bloccati e in attesa**, e **non fanno nulla** — restano fermi indefinitamente aspettando risorse che non otterranno mai. (Distinzione chiave rispetto al livelock: nel deadlock i thread sono *inattivi*, in attesa.)

Definizione — Livelock: simile al deadlock perché i thread coinvolti **non fanno progressi**, e questo danneggia la liveness dell'applicazione. **La differenza cruciale:** nel livelock i thread **non sono inattivi** — sono **impegnati e attivi**, ma le loro azioni **innescano ripetutamente le stesse condizioni**, quindi girano a vuoto senza avanzare.

Esempio reale (molto efficace): due persone si incontrano in un corridoio stretto; ciascuna, per cortesia, si sposta di lato per far passare l'altra — ma si spostano ripetutamente **dallo stesso lato nello stesso momento**, continuando a "ballare" da un lato all'altro senza mai passare.

Il livelock è un rischio con certi algoritmi di **rilevamento e recupero** dei deadlock: se più di un thread reagisce, l'algoritmo può attivarsi ripetutamente. Non è difficile da individuare, perché i thread sono attivi ma bloccati in un ciclo infinito.

Definizione — Starvation (inedia): si verifica quando un thread **non riesce ad accedere** a una risorsa di cui ha bisogno perché altri gliela "sottraggono" costantemente. Il livelock è in effetti un caso particolare di starvation di risorse.

Caso tipico: la starvation colpisce i **thread a bassa priorità** quando la risorsa è molto richiesta dai **thread a priorità più alta**. Il thread a bassa priorità non viene mai "visto"/schedato. Questo danneggia la liveness dell'applicazione, perché anche un thread a bassa priorità ha comunque un lavoro che deve poter essere portato a termine.
